

Архитектурный надзор и управление техническим долгом

AI Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Фомин Дмитрий

Руководитель отдела архитектуры

- Больше 15 лет опыта работы архитектором
- Участие в разработке архитектуры в очень больших проектах (BSS)

 @advoc_diaboly

 fomin.dmitry@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **#OTUS**
AI-Architect-2026-04



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Стратегический фундамент
и планирование проекта

Проектирование и
документирование архитектуры

Продвинутые
архитектурные
паттерны

Инфраструктура

Качество, интеграции
и безопасность

Стратегия, лидерство и
экономика

Проектная работа

ФИНИШ



Маршрут вебинара

Знакомство

Постановка проблемы

Процесс архитектурного надзора

Управление техническим долгом

Что делать с техническим долгом?

Процесс принятия решения

Цели вебинара

К концу занятия вы сможете

Смысл

Для чего вам это уметь

выстраивать процесс контроля
соответствия реализации утверждённой
архитектуре

выявлять, оценивать и
приоритизировать технический долг,
управляя его жизненным циклом

для того, чтобы результат работы
команды не развалился под грузом
проблем

**Как Вы работаете с тех.
долгом?**



Постановка проблемы

Проблема эрозии архитектуры.

- Энтропия: Любая система стремится к хаосу, если не прикладывать энергию для поддержания порядка.
- Причины эрозии:
 - Сжатые сроки (Time-to-market).
 - Смена команды (потеря контекста).
 - "Broken Windows Theory" (Теория разбитых окон): один плохой кусок кода провоцирует других писать так же плохо.
- Симптом: Красивая схема в Confluence больше не соответствует реальности в коде.

Метрики DORA как индикатор проблем

- Как понять, что архитектура сгнила, не глядя в код?
 - Частота релизов падает? Значит, релизить стало больно из-за связности
 - Время внесения изменений растет? Долг замедляет разработку
 - Частота неудачных изменений растет? Хрупкая архитектура
 - Время восстановления растет? Нет наблюдаемости.

Теория разбитых окон в коде

- Если в здании разбито одно окно и его не чинят, вскоре будут разбиты все окна.
- В разработке:
 - Если один разработчик закоммитил код без обработки ошибок и ему это сошло с рук...
 - ...через месяц вся команда перестанет писать try-catch.
- Вывод: Надзор критически важен именно на мелких нарушениях, чтобы сохранить культуру качества.

Вопросы



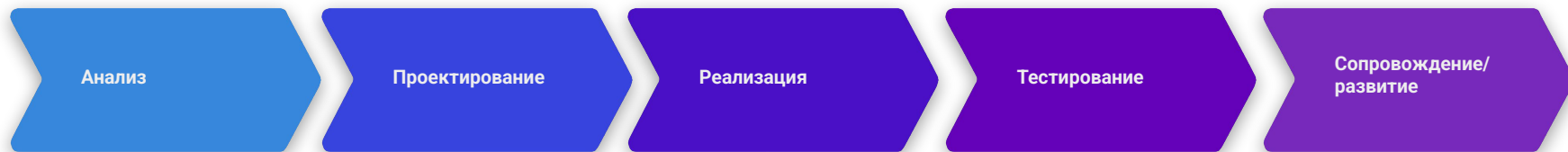
если есть вопросы



если вопросов нет

Процесс Архитектурного надзора

Точки контроля в SDLC



Архитектор валидирует идею (стоит ли вообще это делать?)

Проверка ADR и контрактов API.
Самая дешевая точка исправления ошибки.

Выборочный code-review.
Проверка соблюдения границ модулей.

Приемка функционала.
Автоматические проверки (ArchUnit тесты, Linter).

Мониторинг отклонений (метрики производительности)

Design review

- Цель: Не допустить фундаментальных ошибок до написания первой строки кода.
- Чек-лист (Definition of Ready for Architecture):
 - Есть ли ADR на выбранное решение?
 - Соответствует ли решение NFR (нагрузка, безопасность)?
 - Используются ли технологии из Тех.Политики?
 - Продумана ли схема данных и миграции?

Глубина в Design Review

Неформальный подход

Подсел к коллеге, посмотрел диаграмму. Для мелких задач.

Walkthrough

Автор ведет презентацию, команда задает вопросы.
Риск: автор защищается, команда нападает.

Workshop

Совместное рисование решения у доски. Лучший вариант для сложных систем.

Федеративное управление

- Классический “Архитектор в башне из слоновой кости”, который пытается контролировать всё, тормозит разработку.
- Решение: Делегирование.
 - Centralized: Глобальные стандарты (Non-negotiable). Пример: «Все данные шифруются», «Все LLM вызовы через Gateway».
 - Decentralized: Тактические решения. Пример: «React или Vue?», «Как организовать классы?».
- Принцип: «Centralize Standards, Decentralize Execution».



Концепция “Золотой путь”

- Netflix Paved Road: Культура поощрения, а не принуждения.
- Platform Engineering: Вы предоставляете готовые шаблоны (Scaffolding), где уже настроены линтеры, CI/CD, подключен LLM Gateway и логирование.
- Value Proposition: "Возьми мой шаблон, и ты выйдешь в прод за 2 часа, а не за 2 недели".



Code review

- Фокус внимания архитектора:
 - Leaky Abstractions: Не протекли ли детали реализации базы данных в UI-слой?
 - Новые зависимости: Не добавил ли разработчик библиотеку, которая имеет плохую лицензию или уязвимости?
 - API Contracts: Не сломана ли обратная совместимость?
 - Обработка ошибок: Есть ли retry-политики и circuit breakers там, где они нужны?

Архитектурные фитнес-функции

- Проблема: Архитектура деградирует незаметно. Как измерить "здоровье" системы в динамике? Решение: Автоматические метрики эволюционной архитектуры
- Определение: Тесты, проверяющие, что архитектурные характеристики (производительность, связность, безопасность) не ухудшаются со временем.
 - Latency Fitness: "Ответ модели не должен превышать 500мс в 95% случаев".
 - Data Fitness: "Схема данных в Feature Store не должна меняться без мажорной версии".
- CI/CD Pipeline: Билд падает, если фитнес-функция провалена.

А можно ли тестировать архитектуру?

1.

ArchUnit (Java) (<https://www.archunit.org/>)

```
@Test
public void Services_should_only_be_accessed_by_Controllers() {
    JavaClasses importedClasses = new ClassFileImporter().importPackages("com.mycompany.myapp");

    ArchRule myRule = classes()
        .that().resideInAPackage("..service..")
        .should().onlyBeAccessed().byAnyPackage("..controller..", "..service..");

    myRule.check(importedClasses);
}
```

2.

NetArchTest (.NET) (<https://github.com/BenMorris/NetArchTest>)

```
// Classes in the presentation should not directly reference repositories
var result = Types.InCurrentDomain()
    .That()
    .ResideInNamespace("NetArchTest.SampleLibrary.Presentation")
    .ShouldNot()
    .HaveDependencyOn("NetArchTest.SampleLibrary.Data")
    .GetResult()
    .IsSuccessful;
```

3.

pytest-archon (Python) (<https://pypi.org/project/pytest-archon/>)

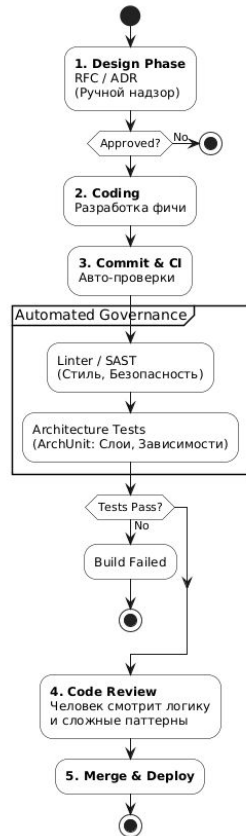
```
from pytest_archon import archrule

def test_rule_basic():
    (
        archrule("name", comment="some comment")
        .match("pytest_archon.col*")
        .exclude("pytest_archon.colgate")
        .should_not_import("pytest_archon.import_finder")
        .should_import("pytest_archon.core*")
        .check("pytest_archon")
    )
```

Линтеры и Статический анализ

- Не тратьте время на ревью стиля кода (PEP8).
- Настройте ruff, sonar или pylint на поиск уязвимостей (Hardcoded secrets, SQL Injection).
- Правило: "Review — для логики и архитектуры, статический анализ — для запятых".

Архитектурный Конвейер (Governance Pipeline)



Вопросы



если есть вопросы



если вопросов нет

Управление техническим долгом

**Создание первого
временного кода — это
как влезание в долги.**

**Гóвард Кáннингем,
программист**

Технический долг – это

это осознанное решение сэкономить время сейчас за счет более сложной работы в будущем

- Как это работает (Финансовая метафора):
 - «Займ»: Выпускаем фичу быстро, срезая углы (без тестов, хардкод, монолит).
 - «Проценты»: Каждое следующее изменение в этом модуле занимает больше времени.
 - «Дефолт»: Разработка останавливается, команда занимается только багфиксингом.



Квадрант Техдолга

По оси X: Намеренный (Deliberate) vs Ненамеренный (Inadvertent).

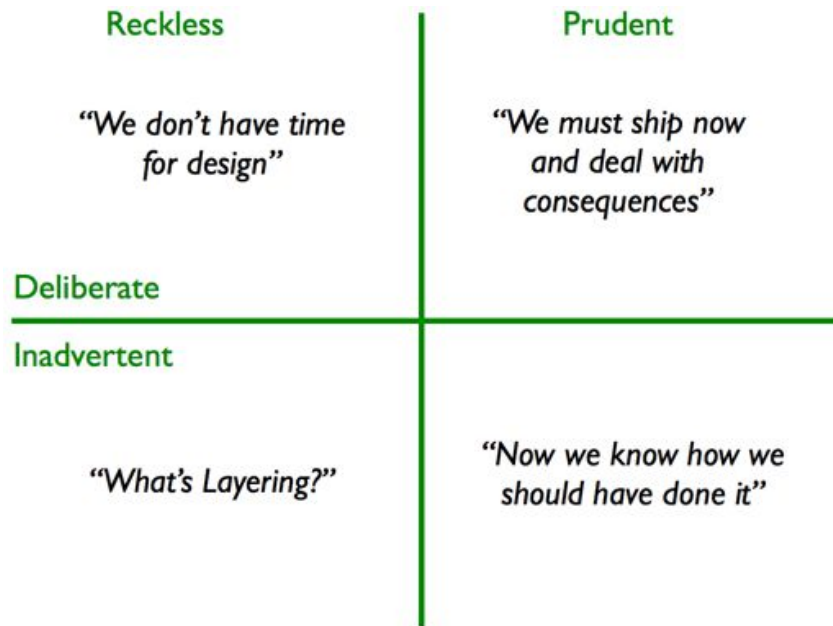
По оси Y: Осмотрительный (Prudent) vs Безрассудный (Reckless).

Осмотрительный + Намеренный: "Нам нужно релизить завтра, мы знаем, что это костыль, мы заводим задачу на исправление". (Это Кредит).

Безрассудный + Намеренный: "У нас нет времени на тесты/архитектуру". (Это Саботаж).

Осмотрительный + Ненамеренный: "Мы узнали, что сделали ошибку, только спустя год обучения". (Это Опыт).

Безрассудный + Ненамеренный: "Что такое слои?". (Это Некомпетентность).



<https://martinfowler.com/bliki/TechnicalDebtQuadrant.html>



Как управлять и визуализировать?

- Учет в Бэклоге:
 - Отдельный тип задач Tech Debt или метка (label).
 - Обязательная оценка (Story Points) — мы должны знать цену возврата.
- Радар техдолга:
 - Визуализация на графике по осям: «Сложность исправления» vs «Влияние на бизнес».
 - Приоритет №1: Легко исправить + Высокое влияние.
- Архитектурный статус:
 - Пометка модулей в документации (цветом на схемах):  Stable,  Refactor needed,  Legacy / Toxic.

Еще один способ посмотреть на радар техдолга

Сложный код/Мало изменений

Сложная ситуация, но редко меняется.

Решение: Поддерживать тестами

Сложный код/Много изменений

Мы постоянно меняем этот кусок, и он ужасно написан. Это генератор багов.

Решение: Рефакторить немедленно

Простой код/Мало изменений

Старое легаси, которое никто не трогает.

Решение: Не трогать! (Оно работает и не просит еды).

Простой код/Много изменений

Хороший код, часто меняется.

Решение: Поддерживать тестами

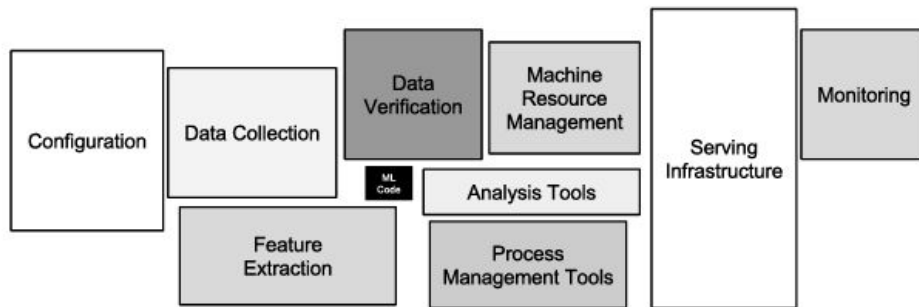
Долг знаний как разновидность тех. долга

- Bus Factor = 1: Если этот человек уйдет (или его собьет автобус), проект встанет.
- Симптомы:
 - «Это знает только Вася, давайте подождем его из отпуска».
 - Нет документации, есть только «племенное знание» (Tribal Knowledge).
- Способы погашения:
 - Парное программирование.
 - Code Review (обязательно другим человеком).
 - Фиксация архитектурных решений в ADR/ADL



Скрытый технических долг в AI

- В классическом IT долг — это "спагетти-код".
- В AI долг — это:
 - Glue Code (Склеивающий код): 90% кода системы — это не математика, а перекладка данных из формата A в формат B.
 - Pipeline Jungles: Пайплайны подготовки данных часто пишутся «на коленке» и становятся неуправляемыми.
 - Dead Experimental Codepaths: Ветки кода от неудачных экспериментов, которые забыли выпилить (и они стреляют в проде).



<https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>

Data Debt

- Unstable Data Dependencies: Модель зависит от данных, которыми вы не владеете (внешний API, другая команда).
- Undeclared Consumers: Кто-то использует выход вашей модели как вход для своей, а вы об этом не знаете.
- Data Drift: Статистические свойства данных изменились, код работает, а бизнес-метрики упали.



Model Debt и принцип CACE

- CACE Principle: Changing Anything Changes Everything. В ML нет изоляции модулей. Изменили одну фичу — поплыли веса всей модели.
- Correction Cascades: Вместо переобучения модели мы пишем фильтры поверх её ответа («Если модель выдала ненормативную лексику — заменить звездочками»).
- Feedback Loops: Модель обучается на данных, которые сама же и сгенерировала (отравление данных).

Вопросы



если есть вопросы



если вопросов нет

Что делать с тех.долгом?

Архитектурные паттерны борьбы с хаосом

- Feature Store: Решает проблему Data Debt. Фичи пишутся один раз, версионизируются и переиспользуются разными моделями. Нет дублирования логики.
- Model Registry: Решает проблему Model Debt. Мы точно знаем, какая версия модели (v1.2) работает в проде, на каких данных она обучена и кто её заапрувил.



Пирамида тестирования в ML

- Data Tests (Great Expectations): Проверяем данные до того, как они попали в модель. Если в колонке age появилось значение -5, пайплайн должен упасть мгновенно.
- Model Tests: Проверка на инварианты. Если мы поменяли пол кандидата в резюме, скор модели изменился? (Fairness/Bias).
- Infrastructure Tests: Проверка того, что модель в докере выдает тот же результат, что и в ноутбуке.



Как "продать" возврат долга бизнесу?

- Бизнес не понимает слова "Рефакторинг" и "Coupling".
- Говорим на языке денег и рисков:
 - Не "У нас плохой код", а "Выпуск новой фичи замедлился в 2 раза (Time-to-market)".
 - Не "Надо обновить библиотеку", а "Есть риск взлома и штрафа (Security)".
- Метрика: Bus Factor (если Вася уйдет, мы встанем на месяц).

Метод RICE для техдолга

- Для того, чтобы говорить на языке денег нужна формула.
- Формула: $(\text{Reach} * \text{Impact} * \text{Confidence}) / \text{Effort}$.
- Адаптация для долга:
 - Reach: Сколько разработчиков страдают от этого кода? (Вся команда или один человек?)
 - Impact: Насколько это замедляет нас? (Сильно/Слабо).
 - Effort: Как долго чинить?

Стратегии погашения

- Правило бойскаута: "Оставь место стоянки чище, чем оно было до тебя". Чиним мелкий долг в рамках текущих задач.
- Налог на техдолг (20% времени): В каждом спринте выделяется квота на инженерные задачи.
- Tech Debt Sprint: Раз в квартал команда занимается только стабилизацией (риск: бизнес ненавидит это).
- Архитектурный субботник: Вся команда собирается на 1 день, чтобы удалить мертвый код.



Когда всё сломалось. Post-mortem

- Правило: Мы не ищем виноватого, мы ищем системную ошибку.
 - Принцип "5 Почему":
 - Упал прод. Почему? -> Out of Memory.
 - Почему? -> Загрузили файл 5ГБ.
 - Почему? -> Нет валидации на входе.
 - Почему? -> В DoD не было пункта про лимиты.
- Результат: Не "Уволить Васю", а "Обновить DoD и добавить алерт".
- Если вы будете ругать за ошибки, люди начнут их скрывать.
Архитектор должен строить культуру безопасности



Вопросы



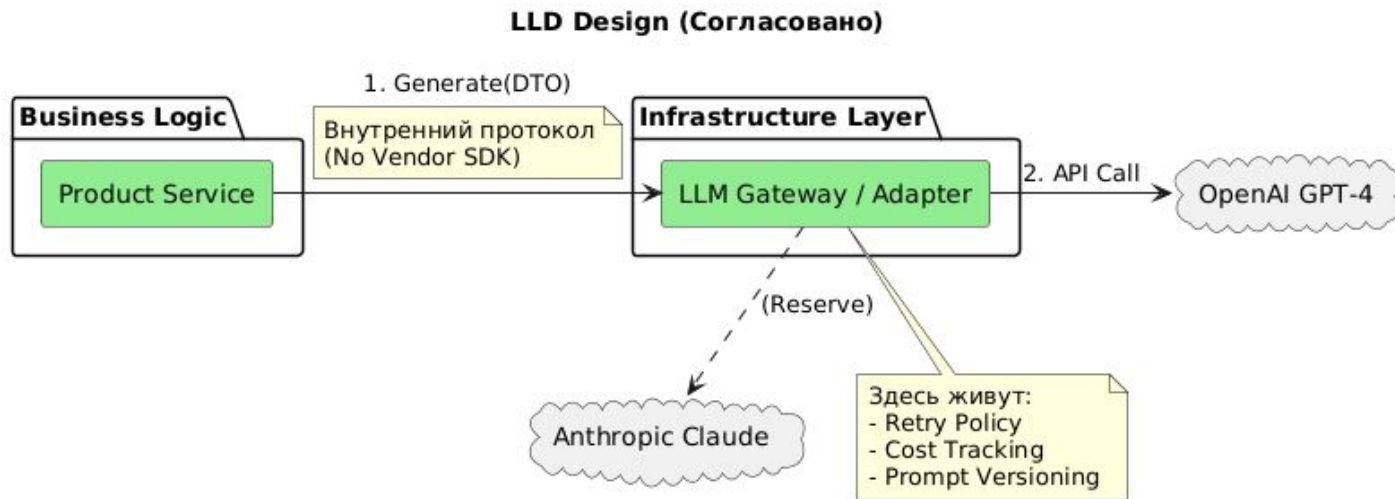
если есть вопросы



если вопросов нет

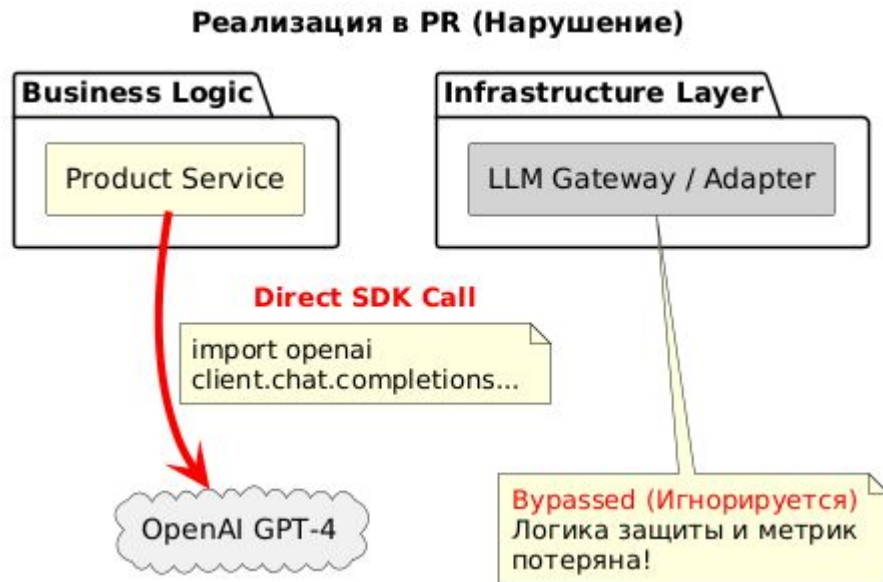
Пример

Кейс для ревью. Как должно быть.



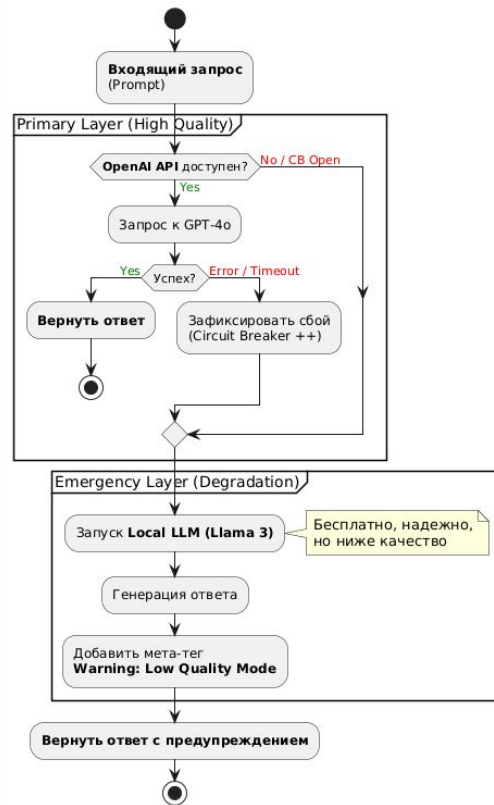
Кейс для ревью. Как получилось

- Аргумент разработчика:
 - «Гейтвей — это оверинжиниринг. Мы точно будем на OpenAI ближайший год. Я убрал лишний слой, стало проще».
- Проблема:
 - Vendor Lock-in: Мы жестко привязались к SDK конкретного вендора.
 - Потеря управления: Логика ретраев и подсчета токенов размазана по бизнес-сервисам.



Паттерн Smart Router: Стратегии отказа

- Принцип Fail Fast (Circuit Breaker):
 - Не ждем таймаутов (30-60 сек). Если уровень ошибок провайдера превышает 10%, шлюз размыкает цепь и мгновенно перенаправляет трафик, не блокируя потоки приложения.
- Иерархия провайдеров (Model Fallback) и Изящная деградация (Graceful Degradation)::
 - Архитектурное решение: «Плохой ответ лучше, чем отсутствие ответа».
 - Переключение на менее "умную", но надежную модель при полном отказе облака.
 - Primary: Максимальное качество (например, GPT-4o).
 - Emergency: Локальная модель или легковесный API (Llama 3 / GPT-4o-mini).



AI FinOps (Управление стоимостью)

- Семантическое кэширование (Semantic Caching):
 - Повторные вопросы («Сколько стоит?» и «Какая цена?») обслуживаются бесплатно и мгновенно (Latency < 20ms).
 - Результат: Снижение затрат на LLM API до 30-40%.
- Token-based Rate Limiting:
 - Классический RPS (Requests Per Second) здесь не работает. Один запрос может содержать 100k токенов.
 - Лимитируем бюджет (USD/Day) или токены (TPM) на каждого клиента/сервис.
- Cost Guardrails (Финансовые предохранители):
 - Hard Limits: При исчерпании бюджета доступ к API блокируется автоматически.
 - Alerting: Уведомление владельцу сервиса при достижении 80% бюджета.

Оформление тех.долга

- Анализ ситуации: Разработчик прав в том, что "сейчас проще", но не прав стратегически.
- Вердикт: ● Переделать.
- Аргументация для команды:
 - Testing: "Как мы будем писать Unit-тесты для ProductService? Нам придется мокать сетевые вызовы OpenAI. С Гейтвеем мы мокаем только интерфейс".
 - Centralization: "Завтра бизнес попросит переключить 10% трафика на дешевую Llama. Без Гейтвея нам придется переписывать 50 сервисов. С Гейтвеем — мы меняем код в одном месте".
- Компромисс (если дедлайн через час):
 - Разрешить временный хардкод, НО:
 - Оформить ADR со статусом "Accepted" (Technical Debt).
 - Создать задачу "Refactor: Introduce LLM Gateway" с блокером на следующий релиз.



Вопросы



если есть вопросы



если вопросов нет

Процесс принятия решений

Architecture Review Board — Добро или Зло?

- Зло: Комитет из 10 седовласых старцев, которые собираются раз в месяц и запрещают всё.
- Добро: Регулярная (еженедельная) встреча для синхронизации лидеров.
- Принцип: "Одобряем стратегию, доверяем тактику". Не надо ревьюить каждую переменную, ревьюим только изменения контрактов и новые технологии.



RFC (Request for Comments) процесс

- Прежде чем писать код или ADR, напиши RFC («Хочу внедрить Redis»).
- Асинхронное обсуждение в Gitlab / Google Docs / Notion.
- Встреча нужна только если есть конфликт мнений.
- Цель: Сдвинуть обсуждение влево (Shift Left) — до начала разработки.



Структура идеального RFC

- Проблема. Какую боль решаем? (Не "Хочу Redis", а "Сессии теряются при редеплое").
- Предлагаемое решение: Суть предлагаемого изменения.
- Альтернативы: Что рассматривали и почему отвергли? (Это фильтр от "вкусовщины").
- Риски и последствия: Что может сломаться? (Если этого раздела нет — RFC заворачивается).
- План обновления: Как переедем без даунтайма?

Ничего не напоминает?



Soft Skills архитектора

- Как сказать «Нет» и не стать врагом народа?
- Метод: Не «Запрещаю», а «Да, но...».
 - «Да, мы можем сделать прямой запрос в OpenAI, но тогда мы потеряем возможность сменить вендора. Давай оценим риски?».
- Надзор — это сервис для команды, а не полиция.

Вопросы



если есть вопросы

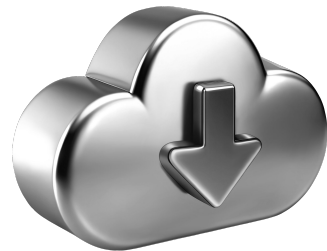


если вопросов нет

Список материалов для изучения

Примечание: Как и для ADR, тема раскрывается в уже упомянутых книгах. Новых монографий именно по архитектурному надзору мало.

1. Tornhill, Adam. 2020. Software Design X-Rays: Fix Technical Debt with Behavioral Code Analysis. Raleigh, NC: Pragmatic Bookshelf.
2. Feathers, Michael C. 2021. Работа с легаси-кодом. Эффективная модификация унаследованного программного обеспечения. М.: Вильямс.



**Подведём
итоги занятия**

Теперь вы сможете:

Отправьте в чат номера достигнутых целей

выстраивать процесс контроля
соответствия реализации утверждённой
архитектуре

выявлять, оценивать и
приоритизировать технический долг,
управляя его жизненным циклом

для того, чтобы результат работы
команды не развалился под грузом
проблем

Вопросы для проверки

1. Архитектор должен просматривать каждый Pull Request в проекте, чтобы гарантировать качество кода и соблюдение стандартов (PEP8/Checkstyle)?
2. Если код написан грязно, без тестов и переменных названы x, y — это Технический долг?
3. Внедрение LLM Gateway — это оверинжиниринг, если мы точно знаем, что будем использовать OpenAI ближайший год?



Ключевые тезисы занятия

1. Надзор — это сервис, а не полиция.
2. Технический долг — это финансовый инструмент.
3. Не будьте живым линтером. Все, что можно проверить автоматически - проверяйте автоматически.



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

